N-DHT — Two-Layer API

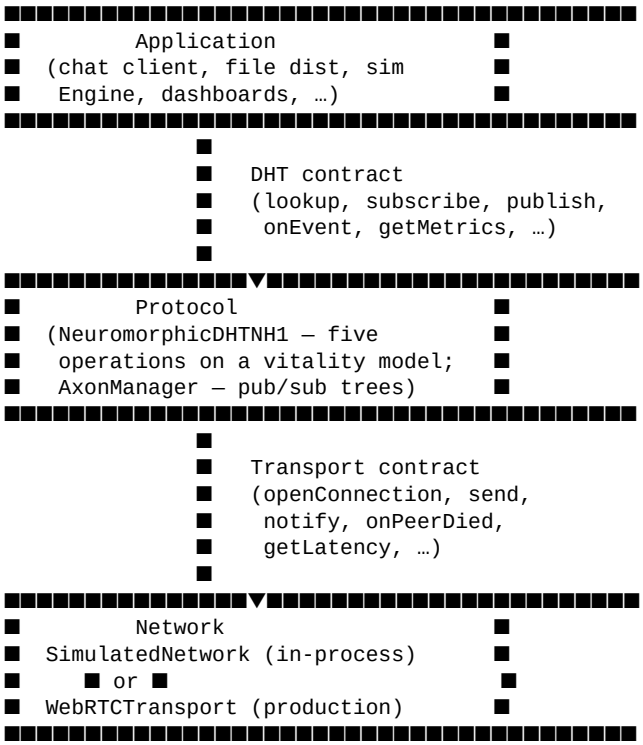
A reference for the contract surface exposed by the protocol. Companion to the implementation punchlist in `documents/implementation/N-DHT-refactor-punchlist.md`*. As of sim v0.70.22 the codebase exposes both layers below; the same protocol code runs in the simulator and in production.*

0. Why two layers

A working distributed hash table has two interfaces, not one. The application above wants to publish, subscribe, and look things up; the network below wants to open a channel, send a message, and notice when a peer goes silent. A protocol is the thing in between — it translates one into the other.

For most of N-DHT's life the simulator collapsed both layers into a single class, because in a single process there's no real difference between "ask the protocol" and "talk to the network". The protocol code reached directly into `nodeMap.get(peerId).synaptome.values()` to make routing decisions, and into `setTimeout(..., latency)` to model wire delay. That made the simulator fast and the code legible, but it also made the protocol *inseparable* from its harness. You could not lift the routing logic out and drop it onto a real WebRTC mesh without rewriting it.

The two-layer split makes the seams explicit. The protocol talks to the application through one contract (called **DHT**) and to the network through another (called **Transport**). The simulator implements the network side in-process; production implements it with WebRTC data channels and WebSocket signaling. The protocol code does not know which.



The horizontal lines are the two contracts. Everything above each line is a consumer of that contract; everything below is an implementation. The protocol sits in the middle and depends only on the

abstractions, not on either concrete side.

1. The DHT contract — facing the application

Defined in `src/contracts/DHT.js`. *Implemented by* `NeuromorphicDHTNH1`.

The application sees one running node. It does not enumerate "all nodes" — it has its own local view, and the DHT is responsible for finding things across the network on its behalf. The contract is small on purpose; eight verbs cover every interaction.

1.1 Lifecycle

```
DHT.start()                    → Promise&lt;void&gt;
DHT.stop()                     → Promise&lt;void&gt;
DHT.join(sponsor)              → Promise&lt;void&gt;
DHT.leave()                    → Promise&lt;void&gt;
```

`start()` allocates the synaptome and spins up the decay tick, the refresh tick, and the heartbeat. The node is ready to join but has no peers yet. `join(sponsor)` opens an initial channel through a `BootstrapEndpoint` (a sponsor URL, a QR-code-pasted pairing string, or a simulator pointer), runs a self-lookup through that sponsor to discover peers near the local ID, then runs the stratified bootstrap that fills one peer per XOR stratum. After `join()` resolves, the synaptome is operational. `leave()` is graceful: notify known peers, close channels, stop heartbeats. `stop()` is the harder version — tear everything down whether peers heard about it or not.

1.2 Operations

```
DHT.lookup(targetKey)                          → Promise&lt;LookupResult&gt;
DHT.subscribe(topicName, handler)              → Promise&lt;Subscription&gt;
DHT.unsubscribe(sub)                           → Promise&lt;void&gt;
DHT.publish(topicName, payload)                → Promise&lt;PublishResult&gt;
```

`lookup` walks the routing layer toward `targetKey`, returning the path it traversed, the hop count, the cumulative observed latency, and a found flag. The walk is an N-DHT routing tick: AP scoring with two-hop lookahead, iterative fallback if no candidate makes XOR progress, and the LEARN side-effects (LTP reinforce, hop caching, triadic introduction, anneal) that follow the path. The application sees one Promise; the protocol underneath fires a chain of `Transport.send('lookup_step', …)` requests, each peer making its own next-hop decision from its own synaptome.

`subscribe` and `publish` ride on the axonal tree primitive. A subscribe is a routed message that walks toward `hash(topic)`; the first axon on the path catches it. A publish reaches the same root and fans out through the tree. Both are async because both can take multiple hops — one Promise per call, no callback forest.

1.3 Identity and observability

```
DHT.getNodeId()                                → bigint
DHT.getSynaptome()                             → SynapseSnapshot[]
DHT.getMetrics()                               → Metrics
DHT.onEvent(handler)                           → unsubscribe()
```

`getNodeId` is the local node's 64-bit identifier. `getSynaptome` returns a read-only snapshot of the routing table — peer ids, weights, latencies, stratum indices. The application is allowed to *look*; it is not allowed to mutate, and the protocol does not consume this surface to drive routing (it has direct access to its own internal state).

`getMetrics` rolls up the same data into the aggregate form the dashboards want: synaptome size, temperature, lookup counters, message-type traffic. Updated continuously, safe to read at any frequency. `onEvent(handler)` registers a listener for the protocol's event stream — the canonical types are `peer-joined`, `peer-left`, `lookup-completed`, `dead-peer-detected`, `anneal-fired`, `pubsub-published`, `pubsub-delivered`, `cycle-snapshot`. Each event is a discriminated-union object with a `type` field; handlers should switch on `type` and ignore unknown values for forward compatibility.

The four observability methods together replace what used to be a god's-eye walk over `nodeMap` from the simulator's Engine. The Engine now subscribes via `onEvent` and reads via `getMetrics`; in production the same contract drives operator dashboards and load-balance plots.

1.4 What the contract forbids

A DHT implementation MUST run as one node. The contract has no method that enumerates "all nodes", no method that takes a peer-id and returns that peer's state, no method that lets the application mutate the routing table. The simulator's Engine creates many DHT instances and orchestrates them — that's a simulator concern, not a protocol concern.

A DHT implementation MUST use only the Transport contract for cross-peer interaction. No direct field access on remote peers; no shared in-process state. This is the rule that makes the same protocol code work in the simulator and on a real network.

2. The Transport contract — facing the network

Defined in `src/contracts/Transport.js`*. Implemented by* `SimulatedNetwork.makeTransport(localNodeId)` *in the sim and by* `WebRTCTransport` *in production.*

One Transport instance per running node. The protocol layer calls *into* it; the protocol does not implement it. Twelve methods, organized into four bands.

2.1 Lifecycle

```
Transport.start(localNodeId)   → Promise&lt;void&gt;
Transport.stop()               → Promise&lt;void&gt;
Transport.getLocalNodeId()     → bigint
```

Same shape as the DHT contract's lifecycle, one level down. `start()` initializes the heartbeat scheduler and the inbound-dispatch tables; `stop()` closes every open channel. Both are idempotent; `start()` after `stop()` is supported.

2.2 Channel pool

```
Transport.openConnection(peerId)    → Promise&lt;boolean&gt;
```

```
Transport.closeConnection(peerId)   → Promise&lt;void&gt;
Transport.isConnected(peerId)       → boolean
```

The protocol declares which peers it wants persistent channels to. The mapping to N-DHT semantics is direct:

> protocol admits peer to synaptome → `openConnection(peerId)` > protocol evicts peer from synaptome → `closeConnection(peerId)`

`openConnection` resolves with `true` if the channel is open and both sides accepted, `false` if the remote refused (the bilateral connection cap is enforced *inside* the transport — the protocol does not need to know how) or is unreachable. Connection-setup cost (WebRTC ICE/DTLS handshake in production, free in the simulator) is paid here, once per peer, not per message.

This shape exists because peer-to-peer routing fails badly when "look at peer X's synaptome" requires opening a fresh channel every time. Persistent channels mean each AP-scoring decision sees an already-warm connection, and the bilateral cap admission is a one-time bilateral handshake rather than an unbounded fan-out.

2.3 Messaging

```
Transport.send(peerId, type, payload)     → Promise&lt;*&gt;     // request/response
Transport.notify(peerId, type, payload)   → Promise&lt;void&gt;  // fire-and-forget
Transport.onRequest(type, handler)        → void           // inbound request
Transport.onNotification(type, handler)   → void           // inbound notify
```

Two outbound primitives: `send` for request/response (the caller needs a return value), `notify` for one-way (the caller doesn't). The split matters because production transports pay round-trip latency for `send` but not for `notify`. The N-DHT protocol uses `send` for the routing-tick chain (`lookup_step`, `lookahead_probe`, `find_closest_set`, `local_probe`) and `notify` for the LEARN side-effects (`reinforce`, `hop_cache`, `lateral_spread`, `triadic_introduce`, `direct_pubsub:*`).

The canonical pattern for a parallel probe is:

```
const results = await Promise.allSettled(
  peers.map(p =&gt; transport.send(p, 'lookahead_probe', { target, fromDist }))
);
```

`Promise.allSettled` (not `Promise.all`) is right because a slow or dead peer in one probe should not fail the whole batch — its rejection is dropped and the rest of the responses still score.

`onRequest` and `onNotification` are how the receiver-side gets its handler invoked. The protocol registers one handler per message type at node-startup time. The `lookup_step` request handler is the one that does most of the work in modern N-DHT routing — it runs the full per-hop logic on the receiver's local state and forwards onward via another `send`.

2.4 Liveness and latency

```
Transport.onPeerDied(handler)        → unsubscribe()
Transport.getLatency(peerId)         → number   // ms, or -1
```

The transport runs a 1 Hz ping/pong heartbeat on every open channel. Each round-trip updates the channel's RTT measurement; missed pongs (heartbeat timeout, default 3 seconds) trigger channel close and an `onPeerDied` event. The protocol consumes `getLatency` to score AP candidates each time they're used in a routing decision, and registers an `onPeerDied` callback to populate a per-node `_deadPeers` set. The candidate-enumeration step in every routing tick filters against that set.

This is the part that finally retired the legacy `nodeMap.get(s.peerId)?.alive` god's-eye liveness check. Production gets exactly the same shape: a peer is alive if its heartbeat is responding; it's dead the moment we miss enough pongs in a row. Same code path in both worlds.

2.5 What the contract forbids

A Transport implementation MUST maintain persistent channels. Channel setup happens once on `openConnection`, not per message. A Transport implementation MUST run a 1 Hz heartbeat on every open channel, expose RTT via `getLatency`, and emit `onPeerDied` on heartbeat timeout. A Transport implementation MUST respect bilateral cap semantics — `openConnection` returns `false` if the remote refused.

The protocol layer is forbidden from reaching around the Transport to access peer state. No `nodeMap.get(peerId)`, no `peer.synaptome.values()` reads, no cross-peer field access. All cross-peer reads happen via `send` / `notify`.

3. BootstrapService — the third, smaller contract

Defined in `src/contracts/BootstrapService.js`*. Implemented by* `SimulatorBootstrapService` *in the sim and by the production rendezvous client.*

Bootstrap is the cold-start problem: a brand-new node has no peers, no synaptome, and no way to find any. The contract is a single method:

```
BootstrapService.bootstrap(sponsor)
  → Promise&lt;{ sponsorId: bigint, transport: Transport }&gt;
```

`sponsor` is a `BootstrapEndpoint` — a discriminated union with three variants:

```
{ kind: 'simulator',   sim: &lt;opaque&gt;, sponsorId: bigint }
{ kind: 'rendezvous',  url: 'wss://…', manifestSig: '&lt;base64&gt;' }
{ kind: 'qr',          sponsorAddr: '&lt;pairing string&gt;' }
```

The simulator variant is an in-process pointer — the simulator's `BootstrapService` opens a channel to that sponsor synchronously. The rendezvous variant points at a WebSocket signaling endpoint with a signed manifest; the production implementation verifies the signature against a known key, opens the WebRTC channel through the signaling server, and discards the rendezvous afterward. The QR variant is for direct device-to-device pairing — no signaling server, no certificate authority, just a sponsor address pasted from a QR scan.

`bootstrap()` returns the sponsor's id and a transport with that one channel open. The DHT's `join(sponsor)` then runs a self-lookup through the sponsor and proceeds with stratified bootstrap on top of the freshly-opened transport.

The split between Transport and BootstrapService matters because production bootstrap is an out-of-band concern (signature verification, manifest fetching, signaling-server handshake) that the routing protocol shouldn't see. Once bootstrap returns, the DHT just has a transport with a channel open; it doesn't know whether that channel came from a QR code or a rendezvous server.

4. Why this shape works

4.1 The same protocol code runs in both worlds

The simulator's `SimulatedNetwork` and the production `WebRTCTransport` both implement the Transport contract. Twelve methods, one signature each. The simulator runs handlers synchronously inside the call (await of a sync return is a microtask no-op); production runs them through WebRTC data channels with real RTT. The protocol does not switch on which transport it's using. Every cross-peer reach goes through `send` or `notify`; every liveness check goes through `_deadPeers` populated by `onPeerDied`.

This is the property that lets the simulator be the deployment vehicle. Years of N-DHT benchmark numbers in the simulator transfer directly: the protocol's hop counts, latency distributions, and pub/sub coverage are the same code path in production.

4.2 Async is the universal acid test

The contract is async-by-default. `lookup` returns a Promise, `subscribe` returns a Promise, `publish` returns a Promise. Even the read-only methods (`getMetrics`, `getSynaptome`) return synchronously, but everything that touches a remote peer is async.

This matters more than it looks. The simulator was historically synchronous because in-process nodes can resolve "send a message" as a function call; production is async because you can't pretend a 200ms WebRTC RTT is free. By insisting on the async signature even in the simulator, the contract forces the protocol code to handle the cases that production exposes — a slow peer, a dropped pong, a peer that died mid-walk — at write time, not at deploy time. The recursive-forwarding `lookup` chain is the most visible example: each receiver awaits the next hop's Promise, so an unreachable peer turns into a rejected Promise, which the chain catches and bubbles back as `exhausted: true`. Production behaves identically.

4.3 Observability is contract-level, not back-channel

Every interesting protocol event — a peer-joined, a peer-left, a lookup-completed, a dead-peer-detected, an anneal-fired — is a `ProtocolEvent` emitted via `onEvent`. The simulator's per-cycle traffic snapshot subscribes to `cycle-snapshot` events instead of walking `nodeMap.values()`. Production dashboards subscribe to the same events.

The benefit isn't just symmetry; it's that the protocol's emitted events become the *test surface*. Smoke tests assert the right events fired the right number of times. Bandwidth-saturation studies measure the per-type distribution of message counters reported by `getMetrics().traffic.byType`. The same numbers that drive the simulator's Gini coefficient calculation drive the production load-balance plot.

4.4 The remaining coupling is sim-only

After the 15-commit migration the only `nodeMap.get(peerId)` sites in the protocol code are:

- `addNode` / `removeNode` — Engine bookkeeping (creating and destroying simulator nodes)

- `bootstrapJoin` — the simulator's god's-eye stratified bootstrap, which production replaces with `BootstrapService.bootstrap(sponsor)` followed by stratified self-lookup
- `lookup(sourceId, …)` — turning the caller's own node id into the local `NeuronNode` that owns the request (V1 only, no V2; production resolves this via the DHT instance owning its own state)
- `_resolveNode` / shim utilities for AxonManager's hex/Node disambiguation — sim-only

None of these are routing-tick V2 violations (cross-peer state reads). The protocol's central algorithm — the recursive-forwarding `lookup_step` chain — runs entirely on receiver-local state, with every cross-peer access going through the Transport contract.

5. Mapping to source files

Contract	Definition	Simulator implementation	Production implementation
DHT	`src/contracts/DHT.js`	`src/dht/neuromorphic/NeuromorphicDHTNH1.js` (multi-node collapse for sim performance)	one instance per node
Transport	`src/contracts/Transport.js`	`src/dht/SimulatedTransport.js` (one per node) backed by `src/dht/SimulatedNetwork.js`	`WebRTCTransport` (planned)
BootstrapService	`src/contracts/BootstrapService.js`	`src/dht/SimulatorBootstrapService.js`	rendezvous + QR variants (planned)
types	`src/contracts/types.js`	shared	shared

The implementation split between the simulator and production is one file replacement: the contracts and the types stay; the Transport's concrete class swaps. The DHT class — the `NeuromorphicDHTNH1` body, all five operations on the vitality model, the AxonManager pub/sub layer — is unchanged.

6. The 15-commit migration in one paragraph

The codebase reached this shape over fifteen commits between sim v0.70.06 and v0.70.22. The early commits (1–3) introduced the contracts and converted Kademlia's iterative lookup to `await transport.send`. Commits 4–9 walked NH-1's routing primitives one at a time onto the Transport: hop-by-hop liveness via `onPeerDied`, two-hop lookahead via parallel `lookahead_probe` RPCs, dead-synapse eviction via `_localCandidate` over `local_probe`, the admission gate via `openConnection` and `getLatency`, and the LEARN side-effects (LTP reinforce, hop caching, lateral spread, triadic introduction) via `notify`. Commit 10 lifted the pub/sub primitives — `routeMessage`, `sendDirect`, `findKClosest` — onto the Transport via `route_msg` request handlers, `direct_*` notifications, and `find_closest_set` RPCs, and made AxonManager async-aware while preserving its sync external API. Commit 11 was the structural endgame for the routing tick: the legacy

source-orchestrated walk (`for hop=0..maxHops: current = nodeMap.get(currentId); …`) became a recursive-forwarding chain where each receiver runs the entire per-hop logic on its own local state and forwards via another `lookup_step`. Commits 12–13 documented the deliberate scope decision to leave NX-15, NX-17, and NX-6 on the legacy path as research/comparison protocols. Commit 14 retired `_nodeMapRef` and the last protocol-layer `nodeMap.get` sites. Commit 15 implemented the DHT contract's observability surface — `getMetrics`, `getSynaptome`, `onEvent`, `snapshotMetrics` — and wired the canonical event-emit sites. The parity-gate benchmark at 25K nodes (post-refactor v0.70.22 vs pre-refactor v0.70.04 reference) confirmed every protocol within the 10% target band.

7. Status

Layer	Status
DHT contract	**defined** (`src/contracts/DHT.js`), implemented by NH-1
Transport contract	**defined** (`src/contracts/Transport.js`), implemented by `SimulatedTransport`
BootstrapService contract	**defined** (`src/contracts/BootstrapService.js`), implemented by `SimulatorBootstrapService`
Production WebRTC transport	planned
Production rendezvous + QR bootstrap	planned

The protocol layer is reusable unchanged in production. The simulator's NH-1 multi-node collapse is a performance affordance, not an architectural requirement — the same `_lookupStep` body, the same `_addByVitality` body, the same AxonManager will run on a per-node basis once the production transport lands.